

Mathable score calculator

September 18, 2024

1 Introduction

The following project presents an automatic scorer system for the game Mathable based on provided images of the board corresponding to each move. The approach is based on object detection using the hsv color space in order to extract the board. Afterwards, the additional border is removed and the resulting board is divided in 196 equal square tiles, as the board size is known to have 14 tiles alongside its width and height. Upon identifying the position and value of the placed token by using the created token templates, the score for each move is computed with respect to the token's numerical value and the constraint, bonus and neighbours of the tile. The score of the turn is considered to be the sum of the scores for each move made during the turn.

2 Approach

2.1 Obtaining a centered image of the board

2.1.1 Idea

I noticed, from the sample images, that the color of the board (Figure 1) is considerably different from the background color, therefore I considered object detection based on the hsv color space a good starting point.



Figure 1: Board

2.1.2 Creating the binary mask

Upon conducting multiple experiments using `cv.getTrackbarPos()` to determine the upper and lower bounds for hue, saturation and value, analysing the resulting binary mask (Figure 12), the final configuration has been set to `lower_bound_hsv = (80, 100, 0)` and `upper_bound_hsv = (120, 255, 255)`. The range of hue values corresponds to a range of blue, as shown in Figure 13.

As some of the previous experimental configurations led to errors when it came to identifying the corners of the board due to noise (such as a white isolated dot higher than the top right corner), I added some erosion to avoid potential such cases by calling `cv.erode(mask_hsv, (10, 10))`.



Figure 2: Board

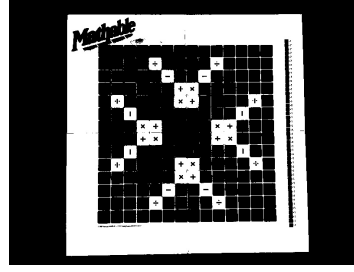


Figure 3: Mask

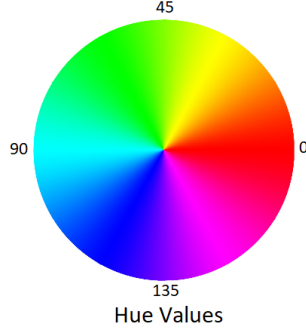


Figure 4: Color wheel

2.1.3 Identifying the corners

The next step consisted in extracting the board based on its corners using the previously obtained mask. As the mask is binary (pixels can take 0 or 255 as value), I considered the board corners to be:

- A (red dot) : The position of the non-zero pixel with the highest value for $x + y$.
- B (green dot) : The position of the non-zero pixel with the lowest value for $x + y$.
- C (yellow dot) : The position of the non-zero pixel with the lowest value for $x - y$.
- D (blue dot) : The position of the non-zero pixel with the highest value for $x - y$.

where x is the column index and y is the row index . A representation of the identified corners is provided in Figure 5.

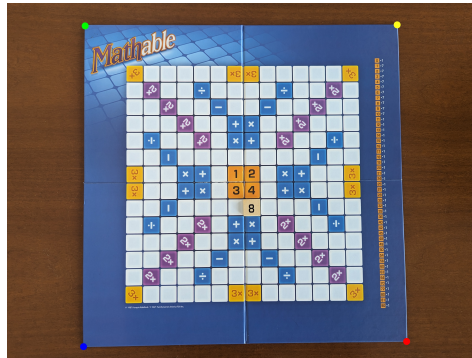


Figure 5: Identifying the corners of the board

2.1.4 Extracting the board in a centered format and removing the border

After identifying the board based on its corners, the next task consisted in centering the result in order to easily and correctly identify the tiles and tokens later on. In order to do so, I used

`cv2.getPerspectiveTransform` and `cv2.warpPerspective` with the previously obtained coordinates of the corners, obtaining results such as the one displayed in Figure 6. The next step consisted in removing the remaining blue border such that I could easily extract the tiles by dividing the board into equally sized squares. I ended up cropping 200 pixels (retained in the `BOARD_BORDER_PX` constant) alongside the width and height. The results are similar to Figure 7.



Figure 6: Result with border

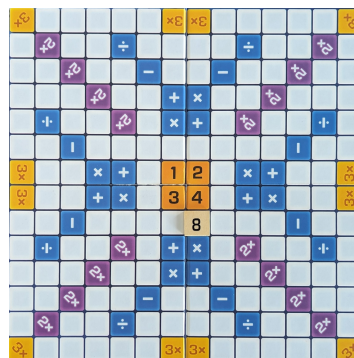


Figure 7: Result without border

2.2 Extracting the tiles from the board

After the centered and cropped image of the board has been obtained, I divided it into 196 images as the board is known to have 14 equal squared tiles alongside its width and height. While studying the misclassifications, I assumed that the fact that some of the token numbers may not be completely covered using the current format of the tiles could be an issue. I then added a padding of 15 for the width and 10 for the height for the extracted tiles, alongside a padding for the whole board as the tiles on the margins encountered issues. This aspect proved to be crucial in correctly identifying the templates in the tiles, which is described in detail in the following section.



Figure 8: Tiles without padding

2.3 Creating templates

In order to create the templates, I used the extracted tiles from one of the up-front provided images which contained all of the available types of tokens (more specifically, `tokens1.jpg`). I then manually cropped the images in order to create valid templates.

This process has been of the "trail and error" kind. The first set of templates was of poor quality due to the fact that the images were not cropped closely enough to the digits, leaving unnecessary padding. The next set of templates failed due to having the single digits cropped very close which resulted in the numbers being confused with the digits, especially if they contained that specific digit (1 instead of 15, for example). The final set of templates has been fine tuned on the training set and is displayed in Figure 9.

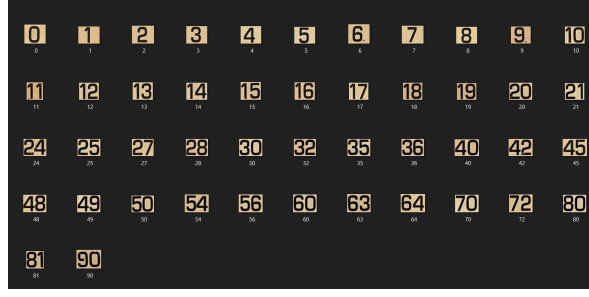


Figure 9: Token templates

2.4 Detection of the position and the token number

The position and the token number of the currently placed tile are detected simultaneously using template matching. The cropped and centered board is extracted from the image corresponding to the current move. The board is later divided into 196 tiles and each tile is compared with all of the token templates in order to obtain their similarity value with $cv.minMaxLoc(cv.matchTemplate(tile, template, cv.TM_CCOEFF_NORMED))$ [1]. The tile that will be considered to correspond to the current move will be the tile with the maximum similarity score such that its position (denoted by i and j) doesn't already have a value for token in the scorer's board. The scorer's board will be updated and the token value for the position of the detected tile will be changed to the detected number (corresponding to the identified template).

The issue this approach presents is that once a position is misidentified, it can never be correctly identified as the board will already have a value for its token. In order to mitigate this potential undesired effect, I added a proofing step that obtains the full token configuration of the board by trying to identify if a token is present (again, with template matching, the template with the maximum similarity is considered, but only if the similarity is higher than a set threshold of 0.7, otherwise the tile will be considered empty). I am then comparing this extracted configuration with the current one, except the newly identified tile. If everything corresponds, the process seems to be going on smoothly. If not, I will compare each to the previous configuration of the board and later on consider the one that has the lowest number of discrepancies. This effort is supposed to offer the chance of a correct detection on a previously misidentified position.

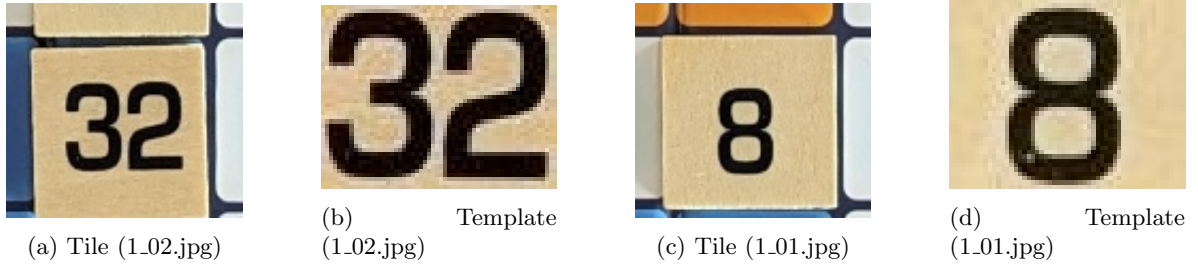


Figure 10: Example of tiles and corresponding detected token template

2.5 Scorer

The scoring implementation is based on instances of two classes: Tile and Scorer.

Tile models a tile of the game with the following proprieties: token (numeric value or None), i (row), j (column), constraint (1 : addition '+', 2 : extraction '-', 3: multiplication 'x', 4: division '/'), bonus (corresponds to the multiplier value, 3 for the 3x tiles, 2 for the 2x tiles and 1 default) .

The scoring logic is modeled with the help of the Scorer class. Among its attributes, it stores a board that is formatted as a dictionary of tuple keys representing the i and j coordinates of the board (similar to a matrix representation), where the values are Tile objects whose values reflect the current configuration of the board.

The number of games is deduced based on the name of the files in the directory that contains

the test images. The same applies for the number of turns. Afterwards, for each game the method *score_game* is called in order to extract for every move the centered and cropped board, detect the placed token based on the logic described in the previous section and compute the score for the move in question. The score follows the rules of the game as it multiplies the token value by the value of bonus and adds the value for each additional vertical or horizontal correct equation that has been formed and satisfies the constraint of the operator ('+', '-', 'x', '/'), if present.

In order to check if the board has been initialized correctly, I created output methods that display the token values, bonus values and constraint values (separately).

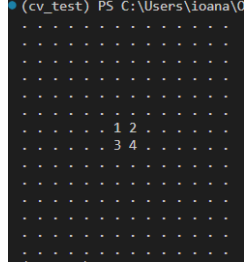


Figure 11: Initial token values

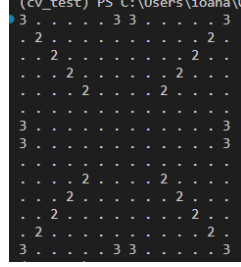


Figure 12: Initial bonus values

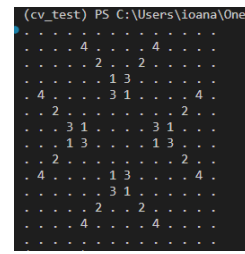


Figure 13: Constraint values

2.6 Experiments

The configuration has been fine-tuned based on training data, obtaining a perfect score upon evaluation on train and test.